

Rename leap to leap_second

[Addresses DE 345](#)

Contents

- [Introduction](#)
- [Wording](#)

Introduction

The LEWG in Belfast requested that I write a paper which provides wording for [DE 345](#) as modified by the discussion in LEWG. The LEWG chose the name `leap_second` as a replacement for `leap`, and identified the need to also change the data member of `tzdb` from `leaps` to `leap_seconds` for consistency.

Wording

1. Modify the synopsis in `[time.syn]` as indicated:

```
...
// 27.11.8, leap second support
class leap_second;

bool operator==(const leap_second& x, const leap_second& y);
strong_ordering operator<=>(const leap_second& x, const leap_second& y);

template<class Duration>
    bool operator==(const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
    bool operator< (const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
    bool operator< (const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
    bool operator> (const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
    bool operator> (const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
    bool operator<=(const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
    bool operator<=(const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
    bool operator>=(const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
    bool operator>=(const sys_time<Duration>& x, const leap_second& y);
template<three_way_comparable_with<sys_seconds> Duration>
    auto operator<=>(const leap_second& x, const sys_time<Duration>& y);
```

2. Modify the synopsis in `[time.zone.db.tzdb]` as indicated:

```
namespace std::chrono {
    struct tzdb {
        string          version;
        vector<time_zone> zones;
```

```

        vector<link>      links;
        vector<leap_second> leap_seconds;

        const time_zone* locate_zone(string_view tz_name) const;
        const time_zone* current_zone() const;
    };
}

```

3. Modify the title of [time.zone.leap] as indicated:

Class leap_second

4. Modify the synopsis in [time.zone.leap.overview] as indicated:

```

namespace std::chrono {
    class leap_second {
    public:
        leap_second(const leap_second&)
        leap_second& operator=(const leap_second&) = default;

        // unspecified additional constructors

        constexpr sys_seconds date() const noexcept;
    };
}

```

5. Modify [time.zone.leap.overview]/p1 as indicated:

1 Objects of type leap_second representing the date of the leap second insertions are constructed and stored in the time zone database when initialized.

6. Modify [time.zone.leap.overview]/p2 as indicated:

2 [Example:

```

    for (auto& l : get_tzdb().leap_seconds)
        if (l <= 2018y/March/17d)
            cout << l.date() << '\n';

```

...

7. Modify [time.zone.leap.overview]/p1 as indicated:

1 Objects of type leap_second representing the date of the leap second insertions are constructed and stored in the time zone database when initialized.

8. Modify [time.zone.leap.nonmembers] as indicated:

```
constexpr bool operator==(const leap_second& x, const leap_second& y) noexcept;
```

1 Returns: x.date() == y.date().

```
constexpr strong_ordering operator<=>(const leap_second& x, const leap_second& y) noexcept;
```

2 Returns: x.date() <=> y.date().

```
template<class Duration>
constexpr bool operator==(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

3 Returns: x.date() == y.

```
template<class Duration>
constexpr bool operator<(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

4 Returns: x.date() < y.

```
template<class Duration>
constexpr bool operator<(const sys_time<Duration>& x, const leap_second& y) noexcept;
```

5 Returns: $x < y.date()$.

```
template<class Duration>
constexpr bool operator>(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

6 Returns: $y < x$.

```
template<class Duration>
constexpr bool operator>(const sys_time<Duration>& x, const leap_second& y) noexcept;
```

7 Returns: $y < x$.

```
template<class Duration>
constexpr bool operator<=(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

8 Returns: $!(y < x)$.

```
template<class Duration>
constexpr bool operator<=(const sys_time<Duration>& x, const leap_second& y) noexcept;
```

9 Returns: $!(y < x)$.

```
template<class Duration>
constexpr bool operator>=(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

10 Returns: $!(x < y)$.

```
template<class Duration>
constexpr bool operator>=(const sys_time<Duration>& x, const leap_second& y) noexcept;
```

11 Returns: $!(x < y)$.

```
template<three_way_comparable_with<sys_seconds> Duration>
constexpr auto operator<=>(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

12 Returns: $x.date() <=> y$.